R4.01 Architecture Logicielle

R4.01 Architecture Logicielle

Date de publication 2026

Table des matières

1. Présentation de la ressource 1
    1. Programme national 1
    2. L'architecture REST 1
    3. Les micro-services 2
    4. Les technologies utilisées 3
    5. Sujet de la SAE ou du projet 3
    6. Utilisations de node à l'IUT 4
    7. Le premier jeu de données utilisé 4
    8. Planning 4
2. Découverte de nodeJS 6
    1. Déclaration des variables 6
        1.1. Cours 6
        1.2. Exercices 8
            1.2.1. Exercice 01 8
            1.2.2. Exercice 02 9
    2. Les prototypes 10
        2.1. Cours 10
        2.2. Exercice 13
    3. Les classes 13
        3.1. Cours 13
        3.2. Exercice 15
    4. Les fonctions fléchées et l'approche fonctionnelle 15
        4.1. Cours 15
        4.2. Exercices 16
            4.2.1. Exercice 01 16
            4.2.2. Exercice 02 16
    5. Les promesses et les fonctions asynchrones 16
        5.1. Cours 16
        5.2. Exercices 20
            5.2.1. Exercice 01 20
            5.2.2. Exercice 02 21
    6. Une spécificité de nodeJS : les événements 21
        6.1. Cours 22
        6.2. Exercice 22
            6.2.1. 22
            6.2.2. Mettre en place les jeux de tests 22
            6.2.3. Compléter la classe stations 23
            6.2.4. Utiliser une bibliothèques csv en événementielle 23
3. Un service proxy 24
    1. Cours 24
        1.1. HTTP 24
        1.2. MongoDB 25
        1.3. Express 27
    2. Exercices 30
        2.1. Exercice 01 30
        2.2. Exercice 02 30
        2.3. Exercice 03 31
        2.4. Exercice 04 31
            2.4.1. Le modèle 32
            2.4.2. Le DAO 32
            2.4.3. Le contrôleur 32
            2.4.4. L'application 32
            2.4.5. La documentation 32
            2.4.6. Pour aller plus loin, la validation des entrées 32
        2.5. Exercice 05 32

4. Un deuxième service avec authentification 34
5. Le point d'entrée de notre application 35

Liste des illustrations

1.1. Architecture microservices (https://microservices.io/) 3
2.1. Promise 17
2.2. Boucle d'événement 22

Liste des tableaux

1.1. Planning prévisionnel 4
3.1. Convention pour les ressources 25

Chapitre 1. Présentation de la ressource

1. Programme national

Le programme national définit un cadre général que nous allons spécialiser

Descriptif

> L’objectif de cette ressource est de présenter des composants de la programmation qui peuvent être utilisés dans plusieurs domaines.

Savoirs de référence étudiés

- Patrons d’architecture (par ex. : Modèle Vue Contrôleur, Model View ViewModel), ...)
- Utilisation de briques logicielles, d’interfaces de programmation, de bibliothèques tierces
- Développement de services web

Nous allons sans un premier temps concevoir des services REST (Representational State Transfer) et les implémenter sous forme d'une architecture de micro services, puis dans un second temps nous allons mettre en place une communication full-duplex par-dessus une connexion TCP, cette approche permet au serveur de notifier des événements aux clients.

2. L'architecture REST

Vous utilisez sans le savoir des services REST au travers de vos applications favorites :

Twitter

> https://developer.twitter.com/en/docs/api-reference-index

Instagram

> https://developers.facebook.com/docs/instagram

Les applications android, web, les clients lourds sollicite des services REST, ces services renvoyant des données bruts pouvant être traitées par les différents langages des applications. Les données sont principalement renvoyées au format JSON (JavaScript Object Notation) ou XML (Extensible Markup Language). Le format JSON sera choisi pour nos services.

A titre d'exemple l'URL suivante permet d'obtenir des informations sur une bière aléatoire : `https://restful-api.dev` au format JSON.

Les attentes d'une API REST doivent répondre aux contraintes architecturales suivante :

Client–serveur

> Nous allons développer dans ce module la partie serveur en nodeJS, la partie cliente sera développez dans les ressources R4.Real.10:Complément web en JS (JavaScript) et R4.Real.11 : Développement pour applications mobiles (Kotlin). Nous utiliseront comme protocole de communication HTTP (Hypertext Transfer Protocol).

Sans état

> Chaque requête devra contenir l'ensemble des informations permettant d'y répondre, les connexions ne seront pas persistantes. L'usage des cookies est par exemple déconseillée et le passage d'un jeton est à préférer.

Avec mise en cache

Les clients et les serveurs intermédiaires peuvent mettre en cache les réponses. Les réponses doivent donc, implicitement ou explicitement, se définir comme pouvant être mise en cache ou non. Dans un soucis de simplification, nous n'implémenterons pas cette fonctionnalité et conserverons le comportement par défaut.

En couches

Le client ne doit pas savoir, si il sollicite un serveur intermédiaire ou un un serveur final. Nous développerons séparément nos services avant de les cacher derrière une passerelle d'API (Application Programming Interface) .

Interface uniforme

Les services REST doivent offrir une interface uniforme, un même service sous une même URL (Uniform Resource Identifier) doit pouvoir offrir différents types de contenus, nous allons offrir uniquement du JSON. Les quatre contraintes de l'interface uniforme sont les suivantes :

Identification des ressources dans les requêtes

La requête peut spécifier le type de ressource attendu

Manipulation des ressources par des représentations

Chaque représentation d'une ressource fournit suffisamment d'informations au client pour modifier ou supprimer la ressource.

Messages auto-descriptifs

Chaque message contient assez d'information pour savoir comment l'interpréter.

HATEOAS (Hypermédia comme moteur d'état de l'application)

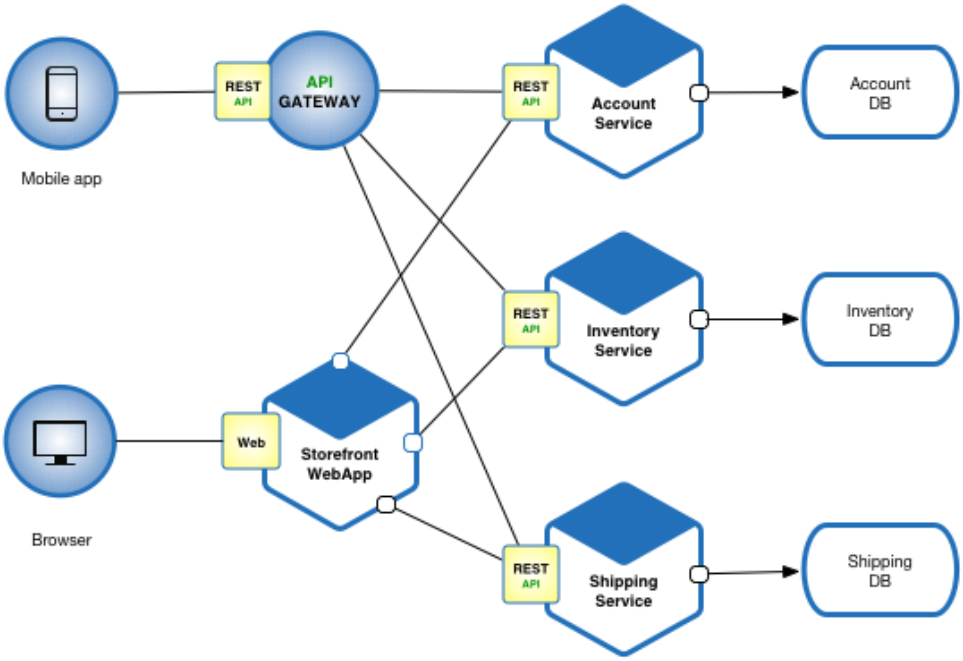
le client doit être en mesure d'utiliser dynamiquement les hyperliens fournis par le serveur pour découvrir toutes les autres actions possibles et les ressources dont il a besoin pour poursuivre la navigation.

3. Les micro-services

Nous allons avoir des services REST implémentés sous forme de microservices. Les microservices s'opposent aux architectures monolithiques. Chaque service doit être le plus simple possible et ne réaliser qu'une fonction, il doit être facilement testables. Les microservices doivent-être élastique (scalable), résiliant (la défaillance d'un service ne doit pas impacter les autres), composable, minimal (les entités et fonctionnalités sont fortement liées), complet (offrir l'ensemble des fonctionnalités).

Nos microservices communiqueront de manière asynchrone en utilisant HTTP.

Figure 1.1. Architecture microservices (https://microservices.io/)



4. Les technologies utilisées

Nous allons réaliser nos services en nodeJS (https://nodejs.org/fr/) une plate-forme logicielle libre en JavaScript, orientée vers les applications réseau événementielles. nodeJS repose sur JavaScript mais il est a noter que la plate-forme ne supporte pas la totalité du langage (https://node.green/), nous utiliserons la version 22 compatible à 99%.

Pour développer la partie serveur HTTP, nous utiliserons le framework express(`https://expressjs.com/`).

Nous utiliserons une version mémoire du Système de Gestion de Base de Données (SGBD) orienté document MongoDB (`https://www.mongodb.com/`) : `https://www.npmjs.com/package/mongodb-memory-server`. Nous utiliserons également mongoose (`https://mongoosejs.com`) un Object Data Manager utilisé comme sur couche sur MongoDB.

5. Sujet de la SAE ou du projet

Vous devrez créer plusieurs micro-services communicants, facilement déployables sur les VM de l'IUT. Chaque micro-service devra être documenté et fourni avec des jeux de tests. Pour la partie serveur HTTP, vous pouvez utiliser le middleware ou le framework de votre choix. Le rendu final sera sur la forme d'une revue de code et une présentation du projet. Vous devrez un jour avant la soutenance avoir déposer sur git votre code, un fichier markdown décrivant l'application et son déploiement et la mise en oeuvre des tests.

A minima devront être créés 3 micro-services répondant à une problématique énoncée :

- Un proxy point d'entré du système
- Un service reposant sur un jeu de données externes de votre choix
- Un service libre

Vous devrez pour chaque service fournir les tests du modèle, des contrôleurs, de l'application ainsi que la documentation. Les alternants pourront se contenter de la documentation et du test applicatif du proxy.

6. Utilisations de node à l'IUT

A l'iut vous utiliserez nvm sous ubuntu[1], un gestionnaire de versions.

Voici les commandes permettant d'installer la Long Time Support :

```
nvm install --lts
nvm use --lts
```

Dans un terminal **node -v** permet d'obtenir la version installée.

La ubuntu ne ne peut accéder à localhost, vous devrez utiliser l'adresse de la ubuntu, i**ip a** vous permet d'obtenir l'adresse.

7. Le premier jeu de données utilisé

Nous allons commercer avec des données de l'OpenData de Nantes (`https://data.nantesmetropole.fr/pages/home/`). Le jeux de donné choisi est `https://data.nantesmetropole.fr/explore/dataset/244400404_disponibilite-temps-reel-velos-libre-service-naolib-nantes-metropole`.

Ce jeux de donnée donne la disponibilité en temps réel des places et des vélos dans les stations vélos en libre-service Naolib de Nantes Métropole. Vous pouvez en avoir un aperçu dans l'onglet données. L'onglet Api permet de réaliser des requêtes et de générer l'URL correspondante.

Trouver la requête permettant de connaître les numéro, l'adresse et le nombre de vélos disponible des stations ayant des vélos disponibles.

```
{
"total_count":108,
"results":[
  {
    "number":95,
    "address":"50, boulevard Michelet",
    "available_bikes":2
  }, ... )]}
```

Cette API rencontre des limites, pouvez vous donner la station contenant un vélo libre la plus proche de votre position, pouvez-vous avoir des stations favorites, la mise à jour étant toutes les cinq minutes, cela faut-il le coût de l'interroger de multiples fois dans ce délai, quel risque peut-on avoir derrière un proxy, ...

Comme toute API elle évolue, voici une version devenue obsolète : `https://data.nantesmetropole.fr/explore/dataset/244400404_stations-velos-libre-service-nantes-metropole-disponibilites/`.

Nous allons répondre à ces problème en réalisant un proxy qui apporte des nouvelles fonctionnalités et un deuxième service pour conserver les favoris.

8. Planning

Voici le planning indicatif :

Tableau 1.1. Planning prévisionnel

semaine	CM	TD	CC	SAE	Objectif
4	1				
5	1	2			Découverte de nodeJS
6	1	3			Création d'un service proxy

[1] `https://github.com/nvm-sh/nvm`

7	1	2	1		Évaluation
10		3			Persistance
11		3			Création d'un service avec authentification
12		2 alternants non présents			Libre
13		3			SAE
14		1	2		Évaluation
15				4?	Projet de SAE

Chapitre 2. Découverte de nodeJS

JavaScript est un langage interprété, faible typé, par références orienté objet à prototype. L'approche prototype est une différence fondamentale,les objets de base ne sont pas des instances de classes. En outre, les fonctions sont des objets de première classe. Le langage supporte le paradigme objet, impératif et fonctionnel.

Vous pourrez exécuter les exemples suivants en utilisant node nom_du_script.

1. Déclaration des variables

1.1. Cours

Les variables globales sont définies sans mot clef, elles sont attachées au scope global, il est déconseillé de les utiliser.

```
//01_global.js

g = 10
console.log(g, typeof g)
//10 number
```

Il est possible de restreindre le langage en utilisant le mode strict, dans ce mode les variables globales ne seront plus utilisées.

```
"use strict"
//g=10 => ReferenceError: g is not defined
```

Vous trouverez sur Internet beaucoup d'exemples reposants sur l'utilisation de var, son utilisation reste déconseillée, les déclarations de variables sont traitées avant que le code soit exécuté, quel que soit leur emplacement dans le code. La portée d'une variable déclarée avec var est le contexte d'exécution courant, c'est-à-dire : la fonction qui contient la déclaration ou le contexte global si la variable est déclarée en dehors de toute fonction.

```
//02_var.js

"use struct"
console.log(a, typeof a)
//undefined undefined
var  a = 44
console.log(a, typeof a)
//44 number
```

Nous utiliserons donc const pour les immuables et let pour les muable, attention pour les objets, const ne fixe que la référence.

```
//03_let_const.js

"use strict"
//c est un nombre constant.
const c = 10
console.log(c, typeof c)
//10 number
//c = 11 => TypeError: Assignment to constant variable

let v = 10
v = 11
console.log(v, typeof v)
//11 number

v="hello"
console.log(v, typeof v)
//hello string

v= function (p) {
    console.log(p)
```

```
}
console.log(v, typeof v)
//[Function: v] function
v(10)
//10

console.log(console)
//Object [console] {
//   log: [Function: log],
//...
console.log(console.log)
//[Function: log]
console.log(console.log("hello"))
//hello
//undefined
```

Comme énoncé précédemment, il est possible de créer des objets sans classe, voici un exemple d'objet littéral.

```
//04_objet_litteral.js

const clio = {
    nom: "Clio",
    marque : "Renault",
    demarree : false,
    equipage : [],
    demarre : function (){
        this.demarree = true
    },
    stop : function (){
        this.demarree = false
    },
    charge : function (passager) {
        this.equipage.push(passager)
    }
}
console.log(clio, typeof clio)
//{
//  nom: 'Clio',
//  marque: 'Renault',
//  demarree: false,
//  equipage: [],
//  demarre: [Function: demarre],
//  stop: [Function: stop],
//  charge: [Function: charge]
//} object

clio.demarre()
console.log(clio)
//{
//  nom: 'Clio',
//  marque: 'Renault',
//  demarree: true,
//  equipage: [],
//  demarre: [Function: demarre],
//  stop: [Function: stop],
//  charge: [Function: charge]
//}
clio.charge("Raoul")
clio.charge({nom: "neuneu", poids : 28});
console.log(clio)
//{
//  nom: 'Clio',
//  marque: 'Renault',
//  demarree: true,
//  equipage: [ 'Raoul', { nom: 'neuneu', poids: 28 } ],
//  demarre: [Function: demarre],
//  stop: [Function: stop],
//  charge: [Function: charge]
//}
console.log(clio.equipage[1].nom)
```

```
//neuneu

clio.poids = 1000
console.log(clio)
//{
//  nom: 'Clio',
//  marque: 'Renault',
//  demarree: true,
//  equipage: [ 'Raoul', { nom: 'neuneu', poids: 28 } ],
//  demarre: [Function: demarre],
//  stop: [Function: stop],
//  charge: [Function: charge],
//  poids: 1000
//}

clio.decharge = function () {
    this.equipage = []
}
clio.decharge()
//{
//  nom: 'Clio',
//  marque: 'Renault',
//  demarree: true,
//  equipage: [],
//  demarre: [Function: demarre],
//  stop: [Function: stop],
//  charge: [Function: charge],
//  poids: 1000,
//  decharge: [Function (anonymous)]
//}

console.log(clio.prototype)
//undefined

console.log(clio)
```

1.2. Exercices

1.2.1. Exercice 01

Dans un même script portant l'extension mjs (module javascript) nous allons reproduire l'objet suivant :

```
{
  total_count: 2,
  results: [
   {
    last_update: "2026-01-20T23:04:41+00:00",
    available_bike_stands: 13,
    number: 19,
    available_bikes: 2,
    name: "SAINT SIMILIEN",
    address: "1, place Saint Similien",
    rental_methods: null,
    bike_stands: 15,
    position: {
     lon: -1.558921,
     lat: 47.219854
    }
   },
   {
    last_update: "2026-01-20T23:04:41+00:00",
    available_bike_stands: 8,
    number: 25,
    available_bikes: 10,
    name: "DALBY",
    address: "61, boulevard Ernest Dalby",
    rental_methods: "creditcard",
    bike_stands: 18,
    position: {
```

```
     lon: -1.530096,
     lat: 47.223919
     }
   }
]}
```

Vous pouvez copier coller des parties de l'exemple précédant pour réaliser les étapes suivantes :

1. Créer deux objets position1 et position2 qui contiendront la première et la deuxième position
2. Créer deux objets result1 et result2 qui contiendront le premier et le deuxième résultat, la date sera stockée en utilisant la méthode Date.parse du "constructeur" Date. (`https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Date/Date`)
3. Créer un tableau results composé de result1 et result2
4. Créer un objet response correspondant à l'objet attendu.

Ajouter à la fin de votre fichier la ligne suivante qui permet d'exporter vos objets :

```
export {position1, position2, result1,result2,results,response}
```

Utiliser le script `exercice01_test.mjs` pour vérifier votre travail.

1.2.2. Exercice 02

Dans nouveau fichier nommé exercice2.mjs, importer response de l'exercice précédant.Vous aller rajouter et tester deux méthodes l'une permettant de rajouter un résultat, et l'autre de supprimer l'ensemble des résultats.

1.2.2.1. La suppression

La méthode doit se nommer clear et ne prend pas de paramètre :

```
response.clear = function () {
    //TODO
}
```

Vous pouvez utiliser this dans la fonction ou response. Le fichier exercice02_test_clear.mjs permet de tester cette étape.

1.2.2.2. L'ajout

Pour la méthode ajout vous prendrez un paramètre, la méthode doit se nommer add.

Les fichiers exericice02_test_add_KO.mjs et exericice02_test_add_KO.mjs, vous permettent de vérifier votre travail.

1.2.2.3. Vérifier les types et le noms

Votre paramètre doit un result avoir les bons attributs et les attributs doivent avoir les bons types. Vous disposez dans le code de deux tableaux firstLevelAttributNames et firstLevelAttributTypes qui contiennent les types attendu. Si le paramètre n'est pas valide, une exception doit-être levée en utilisant une assertion : assert.deepEqual, assertEqual, ...

En JS comme en PHP, le langage étant interprété, il est possible d'accéder à une propriété calculée par exemple :

```
const firstLevelAttributNames= ["number", "name", "address"]
result[firstLevelAttributNames[0]] est équivalent à result.number
```

Object.keys(result) permet d'obtenir un tableau contenant les noms des attributs :

```
assert.deepEquals(Object.keys({number: 1012,name: "01012 - BORNE"}),["number","name"])
est valide
```

Le type peut être vérifié avec typeof, par exemple

```
assert.equal(typeof result.position.lat, "number") est vrai si result.position.lat est un nombre
```

Pour ajouter un élément à un tableau vous pouvez utiliser la méthode push(), la taille d'un tableau est connu avec l'attribut length (`https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array`).

2. Les prototypes

Nous allons brièvement, introduire la notion de prototype.

2.1. Cours

Bien que le mot clef "class" soit disponible en JS, ce n'est qu'un sucre syntaxique. Exception faire des objet littéraux, chaque objet possède une propriété privée qui contient un lien vers un autre objet appelé le prototype. Ce prototype possède également son prototype et ainsi de suite, jusqu'à ce qu'un objet ait null comme prototype. Par définition, null ne possède pas de prototype et est ainsi le dernier maillon de la chaîne de prototype. La majorité des objets JavaScript sont des instances de Object qui est l'avant dernier maillon de la chaîne de prototype.

Les fonctions sont plus que des fonctions, ce sont aussi des constructeurs, elles dispose d'un this. Chaque fonction à par défaut un prototype vide mais qui peut-être enrichi.

Le comportement de console.log() n'est pas décrit par le langage aussi l'affichage peut varier d'une version à l'autre, le problème peut-être contourner avec des librairies comme JSON.stringify() ou util.inspect()

```
"use strict"

const Etudiant = function (nom='John Do',note=0){
    this.nom = nom
    this.note = note

    this.setNote = function (note) {
        this.note = note
    }

}

const etudiant1 = new Etudiant()
console.log("objet",etudiant1)
console.log("typeof",typeof etudiant1)
console.log("instanceof Etudiant", etudiant1 instanceof Etudiant)
console.log("instanceof Object", etudiant1 instanceof Object)
//objet Etudiant { nom: 'John Do',
//  note: 0, setNote: [Function (anonymous)] }
// typeof object
// instanceof Etudiant true
// instanceof Object true

//ajout d'un attrbut en propre
etudiant1.age = 20;
console.log("etudiant1 age en propre", etudiant1)

//etudiant1 age en propre Etudiant {
//   nom: 'John Do',
//   note: 0,
//   setNote: [Function (anonymous)],
//   age: 20
// }

//Le deuxième étudiant n'a pas d'age
const etudiant2 = new Etudiant("etudiant2",20)
console.log("etudiant 2 sans age",etudiant2)

//etudiant 2 sans age Etudiant {
```

```
//   nom: 'etudiant2',
//   note: 20,
//   setNote: [Function (anonymous)]
// }

//Fallback
//le prototype est modifié mais pas l'objet
//l'attribut est en propre n'est pas modifié
Etudiant.prototype.age = 10
console.log("etudiant1 age non modifié",etudiant1)
console.log("etudiant1 prototype",Object.getPrototypeOf(etudiant1))
//etudiant1 age non modifié Etudiant {
//   nom: 'John Do',
//   note: 0,
//   setNote: [Function (anonymous)],
//   age: 20
// }
//etudiant1 prototype { age: 10 }

//ajout du poids au prototype
Etudiant.prototype.poids = 80

console.log("etudiant1 avec poids", etudiant1.poids)
//etudiant1 avec poids 80
console.log(Object.getPrototypeOf(etudiant1))
```

Ce nous venons d'apprendre, un prototype est un objet à partir duquel on crée de nouveaux objets. Dans notre exemple Etudiant est prototype qui va nous permettre de créer des objets comme `etudiant1`.

Il est possible s'enrichir chaque objet avec des méthodes propres, de même il est possible d'enrichir le prototype pour que chaque instance puisse disposer de l'enrichissement.

En JS tous les attributs et les méthodes sont publiques. Il existe un constructeur à la racine : `Object` (`https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Objets_globaux/Object`).

```
"use strict"

//un constructeur comme début de la chaine des prototypes
console.log(Object);
//[Function: Object]
console.log(Object.prototype) //un attribut de la fonction constructrice
//[Object: null prototype] {}
console.log(Object.getPrototypeOf(Object)) //Le prototype
//{}

Object.prototype.age = 10;
//un attribut prototype pour enrichir les prototypes

const Etudiant = function (nom,note) {
    this.nom = nom || 'John Do'
    this.note = note || 0
}

const etudiant1 = new Etudiant();

console.log(etudiant1.age);
//chainage des prototypes
//10
console.log(typeof etudiant1)
//object
```

Voici un nouvel exemple qui illustre la notion de prototype et de chaînage des prototypes :

```
// On commence par créer un objet o pour lequel la fonction f sera
```

```
// son constructeur et lui créera deux propriétés en propre
// a et b :
let f = function () {
    this.a = 1;
    this.b = 2;
}
let o = new f(); // {a: 1, b: 2}

// on ajoute des propriétés au prototype de la fonction
// f
f.prototype.b = 3;
f.prototype.c = 4;

// Note : on ne définit pas le prototype de f avec f.prototype = {b:3,c:4};
// car cela briserait la chaîne de prototype

// o.[[Prototype]] possède les propriétés b and c.
// o.[[Prototype]].[[Prototype]] est Object.prototype.
// Enfin, o.[[Prototype]].[[Prototype]].[[Prototype]] vaut null.
// On a alors atteint la fin de la chaîne de prototype car,
// par définition, null n'a pas de [[Prototype]].
// Ainsi, la chaîne complète est ici :
// {a: 1, b: 2} ---> {b: 3, c: 4} ---> Object.prototype ---> null

console.log(o.a); // 1
// Existe-t-il une propriété 'a' en propre sur o ? Oui, elle vaut 1.

console.log(o.b); // 2
// Existe-t-il une propriété 'b' en propre sur o ? Oui, elle vaut 2.
// Le prototype possède également une propriété 'b' mais elle n'est pas
// utilisée.
// C'est ce qu'on appelle l'ombrage (shadowing en anglais)

console.log(o.c); // 4
// Existe-t-il une propriété 'c' en propre sur o ? Non, on vérifie le
// prototype.
// Existe-t-il une propriété 'c' en propre sur o.[[Prototype]] ?
// Oui, elle vaut 4.

console.log(o.d); // undefined
// Existe-t-il une propriété 'd' en propre sur o ? Non, on vérifie le
// prototype.
// Existe-t-il une propriété 'd' en propre sur o.[[Prototype]] ? Non, on vérifie le
// prototype.
// o.[[Prototype]].[[Prototype]] est Object.prototype et ne contient pas
// de propriété 'd' par défaut. On vérifie son prototype.
// o.[[Prototype]].[[Prototype]].[[Prototype]] est null, on arrête la recherche
// aucune propriété n'est trouvée, le moteur renvoie undefined.
```

Cela commence à devenir compliqué, de plus il est possible de passer une instance d'un objet à un autre constructeur et définir une forme d'héritage, dans l'exemple qui suit Object.create (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/create) permet de créer un objet à partir d'un prototype, call() (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call) permet d'appeler une autre fonction en passant son this en paramètre et Object.prototype.constructor (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/constructor) permet de définir le constructeur.

```
"use strict"
// Forme, la classe parente
let Forme = function() {
    this.x = 0;
    this.y = 0;
}

// Méthode de la classe parente
Forme.prototype.deplacer = function(x, y) {
    this.x += x;
    this.y += y;
    console.info('Forme déplacée.');
```

```
};

// Rectangle - classe fille
function Rectangle() {
    // on appelle un autre constructeur pour initialiser this
    Forme.call(this);
}

// La classe fille surcharge la classe parente
Rectangle.prototype = Object.create(Forme.prototype);
Rectangle.prototype.constructor = Rectangle;

var rect = new Rectangle();

console.log('instance de Rectangle ? ', (rect instanceof Rectangle));
// true
console.log('une instance de Forme ? ', (rect instanceof Forme));
// true
rect.deplacer(1, 1);
// Affiche 'Forme déplacée.'
```

2.2. Exercice

Créer un fonction permettant de définir un animal caractérisé par son nom et disposant de la méthode crie qui renvoie le nom suivi de "crie " puis une classe chien dont le crie renvoie nom suivi de "crie wawa ". Pour appeler la fonction crie de Animal vous pourrez utiliser :

```
 Animal.prototype.crie.call(...)
```

3. Les classes

3.1. Cours

Nous allons par habitude privilégier les sucres syntaxiques qui vont suivre.

Le mot clef class permet de définir une classe et constructor le constructeur. La notion de surcharge n'existe pas.

```
class Rectangle {
  constructor(hauteur, largeur) {
    this.hauteur = hauteur;
    this.largeur = largeur;
  }
}
```

Des classes peuvent-être anonyme ou nommée.

```
// anonyme
let Rectangle = class {
  constructor(hauteur, largeur) {
    this.hauteur = hauteur;
    this.largeur = largeur;
  }
};

// nommée
let Rectangle = class Rectangle {
  constructor(hauteur, largeur) {
    this.hauteur = hauteur;
    this.largeur = largeur;
  }
};
```

Les classes permettent de définir des getter et des setter sur les attributs (get nom_attribut, set nom_attribut).

```
class Rectangle {
    constructor(hauteur, largeur) {
```

```
        this.hauteur = hauteur;
        this.largeur = largeur;
    }

    get area() {
        return this.calcArea();
    }

    set setLargeur(largeur) {
        if (largeur < 0)
            largeur = -largeur
        this.largeur = largeur
    }

    calcArea() {
        return this.largeur * this.hauteur;
    }
}

const carré = new Rectangle(10, 10);
carré.setLargeur = -10;
console.log(carré.area);
```

Les attributs peuvent être définis explicitement et être privés (#).

```
class Rectangle {
  #height = 0;
  #width;
  constructor(height, width) {
    this.#height = height;
    this.#width = width;
  }
}
```

Le mots clefs extend permet l'héritage simple, la notion d'interface n'existe pas. super permet de référencer les méthodes ou les attributs de la classe mère, les redéfinitions sont possibles et le polymorphisme.

```
class Cat {
    constructor(name) {
        this.name = name;
    }

    speak() {
        console.log(`${this.name} makes a noise.`);
    }
}

class Lion extends Cat {
    speak() {
        super.speak();
        console.log(`${this.name} roars.`);
    }
}

const l = new Lion("Fuzzy");
l.speak();
// Fuzzy makes a noise.
// Fuzzy roars.
```

Le mots clef static permet de définir des méthodes statiques ou des attributs statiques.

```
class ClassWithStaticMethod {
  static staticProperty = 'someValue';
  static staticMethod() {
    return 'static method has been called.';
  }
  static {
    console.log('Class static initialization block called');
  }
}
```

```
console.log(ClassWithStaticMethod.staticProperty);
// expected output: "someValue"
console.log(ClassWithStaticMethod.staticMethod());
// expected output: "static method has been called."
```

Des spécificités comme les fonctions génératrices existent, nous les introduirons au fur et à mesure des besoins (https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/function*).

3.2. Exercice

Chaque classe devra être dans un fichier séparé de même que les tests. Définir une classe Humain définie par un nom qui ne peut être modifié mais qui peut-être consulté. Une classe Etudiant qui représente un Humain augmenté d'une note, une Promotion nommée qui contient des étudiants. On souhaite pouvoir connaître le nombre d'instance d'Humain. Vous pourrez utiliser throw new Error("...") et un try catch pour interdire l'ajout d'autre chose qu'un étudiant à la promotion.

4. Les fonctions fléchées et l'approche fonctionnelle

4.1. Cours

Les fonctions fléchées (`()=>{}`) ne sont un raccourci syntaxique ce sont des fonctions sans `this`, voici un exemple d'utilisation :

```
class Humain {
    constructor(sex,age){
        this.sex=sex;
        this.age=age;
    }
    toString(){
        const mensonge = function(){
            console.log(this);
            //probleme est que le this n'est pas celui de Humain
            //mais celui de mensonge
            return 20;
        }
        return "Humain "+this.sex+" "+mensonge();
    }
}

const robert = new Humain("F",18);
console.log(robert.toString());
//undefined
//Humain F 20

//Une version avec une fonction flechee qui na pas this
class Humain2 {
    constructor(sex,age){
        this.sex=sex;
        this.age=age;
    }
    toString(){
        const mensonge = () =>{
            console.log(this);
            //le this est celui de Humain2, mensonge n'en a pas
            this.age -=10;
            return this.age;
        }
        return "Humain "+this.sex+" "+mensonge();
    }
}
const robert2 = new Humain2("F",18);
console.log(robert2.toString());
console.log(robert2.age);
//Humain2 { sex: 'F', age: 18 }
//Humain F 8
```

```
//8
```

L'opérateur ... permet de transformer une collection en un ensemble de paramètres.

```
"use strict"
const tab = [1,2,3]
console.log(tab)
//[ 1, 2, 3 ]
console.log(...tab)
//1 2 3
```

4.2. Exercices

Vous devez utiliser la documentation de Array (https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array).

4.2.1. Exercice 01

En partant du tableau suivant et en utilisant map de Array ainsi que l'opérateur ternaire (cond?retour_si_vrai:retour_si_faux) augmenter les notes sans dépasser 20. Puis calculer la moyenne en utilisant reduce et enfin donner le nombre de notes supérieures à la moyenne.

```
const tab = [10,20,0,5,8,12]
```

4.2.2. Exercice 02

Reprendre la promotion pour n'ajouter un élément que si il n'est pas présent, supprimer un étudiant, donner la moyenne, la liste des étudiant ayant la moyenne et ceux qui ne l'ont pas.

5. Les promesses et les fonctions asynchrones

Par défaut en JS toute fonction est synchrone, cela signifie que lors de son appel, elle s'exécute immédiatement et que l'instruction suivante attend la fin de l'instruction précédante avant se s'exécuter. Lorsque l'on manipule des entrées/sorties ou des appels réseau, ce comportement peut-être préjudiciable, par exemple si votre navigateur exécute un appel synchrone, il ne peut réagir aux événements.

5.1. Cours

Une première approche est l'utilisation de callback. Le principe de fonction de callback est la manière la plus classique de procéder:

1. on appelle une fonction asynchrone (A) en lui passant une autre fonction (B) – appelée fonction de callback – en paramètres;

2. la fonction asynchrone (A) rend immédiatement la main au programme;

3. la fonction de callback (B) sera appelée une fois que l'opération aura terminé son exécution.

Ainsi, quand on appelle plusieurs fonctions asynchrones d'affilée, leurs fonctions de callback respectives ne seront pas forcément exécutées dans le même ordre[1].

```
// setTimeout() est une fonction asynchrone qui exécute la fonction de callback
// après quelques millisecondes d'attente
setTimeout(() => console.log('a'), 50); // afficher a dans 50 millisecondes
setTimeout(() => console.log('b'), 90); // afficher b dans 90 millisecondes
setTimeout(() => console.log('c'), 20); // afficher c dans 20 millisecondes
// => ordre d'affichage: c, a, puis b
//    car les opérations asynchrones s'exécutent en parallèle
console.log("debut")
//s'affiche avant la fin des timers
```

[1] source : https://adrienjoly.com/cours-nodejs/sync-vs-async.html

En nodeJs, vous trouverez les conventions suivantes pour les callBack, la fonction de callback prend généralement deux paramètres:

- le premier paramètre est une instance de la classe Error – si l'opération a échoué – ou null
- le deuxième paramètre est le résultat de l'exécution de l'opération, dans le cas où elle s'est exécutée sans erreur. C'est la valeur que l'on aurait passé à return si notre fonction était synchrone.

```
// Ne disposant pas de mogodb, vous ne pourrez exécuter ce code
// Cette fonction appelera la fonction callback pour transmettre une réponse ou une erreur.

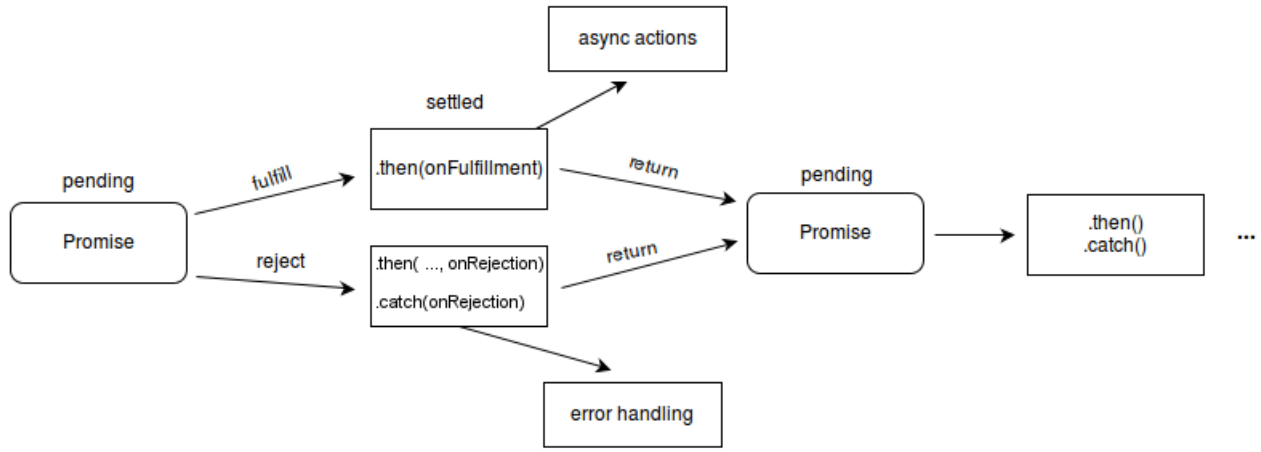
const meaningOfLife =  (callback) => {
    // supposons que la réponse est disponible dans une collection mongodb
    collection.findOne({ meaning: 'life' },  (err, res) => {
        if (err) {
            // la requête db a échoué => appeler callback en incluant le message d'erreur
            callback(new Error('meaningOfLife failed because ' + err.message));
        } else {
            // la requête db à réussi => appeler callback en incluant la réponse
            callback(null, res.answer);
        }
    });
}

meaningOfLife( (err, answer) =>{
    if (err) {
        console.error('meaningOfLife() a rapporté une erreur:', err);
    } else {
        console.log('valeur retournée par meaningOfLife():', answer);
    }
});
```

L'approche callback permet de cascader des appels en passant dans la callBack une autre callback et ainsi de suite (callback hell). Une autre impossibilité est la synchronisation des appels asynchrone, je souhaite attendre la fin de d'appels asynchrone avant de progresser, comment faire. Les promesses (Promise) permettent de répondre à ce problème, les fonction asynchrone (async/wait) sont un nouveau sucre syntaxique sur les promesses.

Une promesse est un objet (Promise) qui représente la complétion ou l'échec d'une opération asynchrone. Une promesse représente une valeur qui peut être disponible maintenant, dans le futur voire jamais.

Figure 2.1. Promise



Une Promise peut être:

resolve

si l'opération asynchrone s'est exécutée avec succès;

reject

si une erreur est survenue pendant l'exécution de cette opération.

Les fonctions .then() et .catch() permettent de définir le comportement à adopter si la Promise est résolue ou rejetée, respectivement. .finally() est exécuté que la promesse soit résolue ou non.

Le promesse suivante tire un nombre aléatoire compris entre 0 et 1, si 0 est tiré, elle renvoie "gagné", "perdu" sinon.

```
"use strict"
const unePromesse = new Promise((resolve,reject) =>{
    if (Math.floor(Math.random() * 2)==0)
        resolve("gagné")
    else
        reject("perdu")
})

unePromesse.then(res=>console.log(res)).catch(res=>console.log(res))
console.log("début")
//début
//perdu ou gagné
```

Jeux est une fonction qui renvoie une promesse, jeux prend deux paramètres nombre, le nombre joué et plage la plage du tirage entre 0 et plage-1.

```
const jeux = (nombre,plage) =>  new Promise((resolve,reject) =>{
    if (Math.floor(Math.random() * plage)==nombre)
        resolve("gagné")
    else
        reject("perdu")
})

jeux(0,1).then(res=>console.log(res)).catch(res=>console.log(res))
console.log("début")
//début
//gagné
```

Il est possible d'ordonner les promesses en séquence en renvoyant une nouvelle promesse dans le then, le catch ou le finally.

```
"use strict"
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})

timer(10)
    .then(s => {console.log(s); return timer(20)})
    .then(s => {console.log(s)})
```

Promise dispose de méthodes statiques qui permettent d'exprimer d'autres formes de synchronisation :

Promise.all(iterable)

Renvoie une promesse tenue lorsque toutes les promesses de l'argument itérable sont tenues ou une promesse rompue dès qu'une promesse de l'argument itérable est rompue. Si la promesse est tenue, elle est résolue avec un tableau contenant les valeurs de résolution des différentes promesses contenues dans l'itérable (dans le même ordre que celui-ci). Si la promesse est rompue, elle contient la raison de la rupture de la part de la promesse en cause, contenue dans l'itérable. Cette méthode est utile pour agréger les résultats de plusieurs promesses tous ensemble.

Promise.allSettled(iterable)

Attend que l'ensemble des promesses aient été acquittées (tenues ou rompues) et renvoie une promesse qui est résolue après que chaque promesse ait été tenue ou rompue. La valeur de résolution de la promesse renvoyée est un tableau dont chaque élément est le résultat des promesses initiales.

Promise.any(iterable)

Renvoie une seule promesse dont la valeur de résolution est celle de la première promesse résolue de l'itérable passé en argument.

Promise.race(iterable)

Renvoie une promesse qui est tenue ou rompue dès que l'une des promesses de l'itérable est tenue ou rompue avec la valeur ou la raison correspondante.

Promise.reject(raison)

Renvoie un objet Promise qui est rompue avec la raison donnée.

Promise.resolve(valeur)

Renvoie un objet Promise qui est tenue (résolue) avec la valeur donnée. Si la valeur possède une méthode then, la promesse renvoyée « suivra » cette méthode pour arriver dans son état, sinon la promesse renvoyée sera tenue avec la valeur fournie. Généralement, quand on veut savoir si une valeur est une promesse, on utilisera Promise.resolve(valeur) et on travaillera avec la valeur de retour en tant que promesse.

```
"use strict"
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})

Promise.race([timer(10),timer(20)]).then(s => console.log(s))
//fini après 10
Promise.any([timer(10),timer(20)]).then(s => console.log(s))
//fini après 10
Promise.all([timer(10),timer(20)]).then(s => console.log(s))
//[ 'fini après 10', 'fini après 20' ]
```

Une fonction asynchrone est une fonction précédée par le mot-clé async, et qui peut contenir le mot-clé await. async et await permettent un comportement asynchrone, basé sur une promesse (Promise), écrite de façon simple, et évitant de configurer explicitement les chaînes de promesse. Il est à noter qu'un module est une fonction asynchrone.

```
//fichier .js
"use strict"
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})

const f = async () => {
   console.log(await timer(20))
    console.log(await timer(10))
}
console.log("début")
f()

//début
// fini après 20
// fini après 10
```

```
//fichie .mjs module
"use strict"
const timer = (temps) =>  new Promise((resolve) =>{
    setTimeout(() => resolve("fini après " + temps), temps)
})


   console.log(await timer(20))
    console.log(await timer(10))

console.log("début")
```

```
// fini après 20
// fini après 10
// /!\ début
```

5.2. Exercices

Les timers étant limités, nous allons utiliser des bibliothèques nodeJS, il nous faut commencer par créer un projet. Créer un répertoire s'y placer et taper :

```
npm init
```

Vous obtiendrez un fichier package.js qui définie votre projet, comme tout projet, il vous faudra travailler à la racine.

```
{
  "name": "project-name",
  "version": "0.0.1",
  "description": "Project Description",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "repository": {
    "type": "git",
    "url": "the repositories url"
  },
  "author": "your name",
  "license": "N/A"
}
```

5.2.1. Exercice 01

fecth() est un client HTTP fourni avec JS mais selon la version non prévu pour franchir un proxy, nous allons utiliser node-fecth quie l permet.

Rajouter dans script "start": "node index.mjs" et créer un fichier `index.mjs`. Exécuter votre script vide en utilisant **npm run start** ou **npm start**.

```
//package.json
"scripts": {
    "start": "node index.mjs
  }
```

Nous allons ajouter le module nodeJS, la commande permet d'ajouter le module au projet courant[2] :

```
npm install node-fetch
```

Observer la modification du package.json et la création du répertoire node_modules, supprimer le répertoire node_modules et lancer :

```
npm install
```

Vous venez d'observer que vous pouvez recréer les dépendances de vos projets et que le répertoire node_modules n'est jamais à fournir.

Tester avec l'explorateur web d'API (onglet API), l'API des disponibilités en temps réel des places et des vélos dans les stations vélos en libre-service Naolib de Nantes : https://data.nantesmetropole.fr/explore/dataset/244400404_disponibilite-temps-reel-velos-libre-service-naolib-nantes-metropole.

Nous allons chercher à obtenir la liste des stations acceptant les cartes bleue contenant un vélo libre triée par distance croissante par rapport à la postion 47.216671 Y: -1.55.

Étant à l'IUT, vous devrez utiliser un proxy, installer https-proxy-agent.

Comprendre et adapter le code suivant :

[2]Si le télécharge ment bloque autoriser http avec npm config set strict-ssl false

```
"use strict"
import fetch from 'node-fetch';
import {HttpsProxyAgent} from 'https-proxy-agent';

const proxy = process.env.https_proxy

let agent = null
if (proxy != undefined) {
    console.log(`Le proxy est ${proxy}`)
    agent =  new HttpsProxyAgent(proxy);
}
else {
    //pour pouvoir consulter un site avec un certificat invalide
    process.env.NODE_TLS_REJECT_UNAUTHORIZED = "0";
    console.log("Pas de proxy trouvé")
}

const urlBase = 'https://jsonplaceholder.typicode.com/'
const url = urlBase+"posts"

let response = agent!=null ? await fetch(url, {agent: agent}):await fetch(url)
let json = await response.json()
console.log(json)
```

Le obtenu doit ressembler au suivant :

```
[
  {
    name: '003-STRASBOURG',
    address: '4, rue des Carmélites - Place Tirant Lo Blanc',
    position: { lon: -1.551979, lat: 47.21682 }
  },
...]
```

Avons nous obtenu, l'ensemble des résultats, modifier votre requête pour obtenir toutes les stations contenant au moins un vélo ?

Pour récupérer l'ensemble des stations vous devrez faire au moins deux requêtes, la première avec une "limit" de -1, vous permet de découvrir le nombre de station et de récupérer les 100 premières, les autres sont à renouveler avec un "offset".

5.2.2. Exercice 02

NodeJS est fourni avec un ensemble de modules (https://nodejs.org/api/) , nous allons utiliser File System. Le code suivant permet de en utilisant readdir de lister un répertoire en utilisant une promesse readdir.

```
"use strict"

import { readdir } from 'node:fs/promises';

try {const files = await readdir(".")
   files.forEach(file => console.log(file))

} catch (err) {
    console.error(err);
}
```

Mais readdir existe aussi sous forme de callback en ayant importé readdir depuis node:fs reproduire l'exemple précédant :

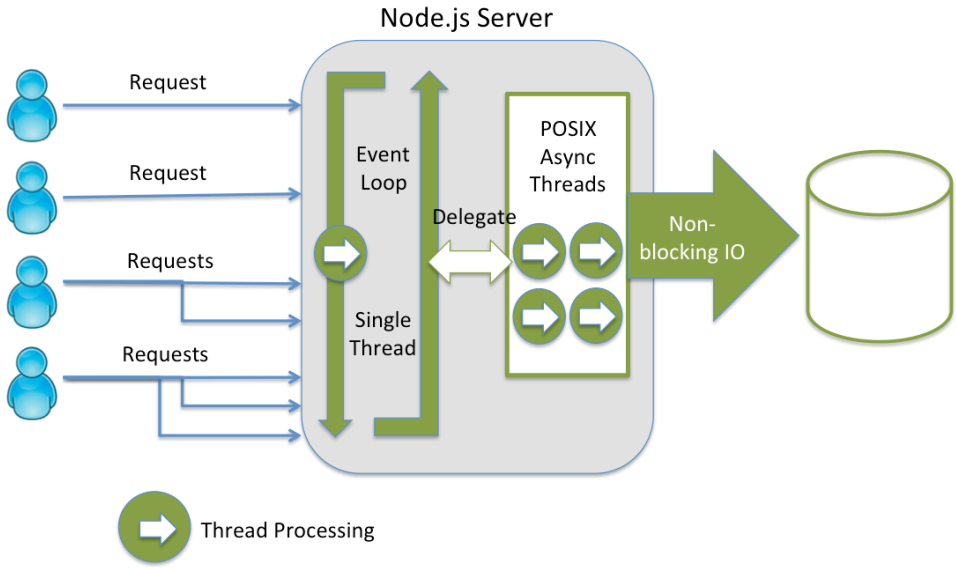
```
import { readdir } from 'node:fs';
```

6. Une spécificité de nodeJS : les événements

Node.js est monothreadé : toutes les opérations sont exécutées dans un seul thread. Mais Node.js s'appuie sur la boucle d'événements (event loop) qui lui permet de traiter des requêtes asynchrones non bloquantes. Ce fonctionnement permet à Node.js de réaliser des opérations d'entrées-sorties non bloquantes malgré son caractère mono-

threadé. Ces opérations peuvent être, par exemple, des requêtes HTTP, des lectures ou écritures de fichiers, des requêtes en base de données, etc. Les opérations synchrones sont exécutées séquentiellement en exploitant la pile d'appels (call stack). Tant que des opérations sont présentes sur la pile d'appels celles-ci sont exécutées. Il est donc essentiel de ne pas avoir des opérations qui monopolisent cette pile puisqu'il n'y a qu'un seul thread. Les opérations les plus pénalisantes doivent donc être exécutées de manière asynchrone. Elles sont alors gérées par l'event loop qui communique avec le noyau du système d'exploitation hôte pour lui déléguer l'exécution de ces opérations asynchrones dans des threads système[3].

Figure 2.2. Boucle d'événement



6.1. Cours

L'objet EventEmitter dispose de deux méthodes : on pour capturer et emit pour émettre.

```
import events from "events"

const em = new events.EventEmitter();

em.on('firstEvent', function (data) {
    console.log(data);
});

em.on('secondEvent', function (data) {
    console.log(data);
});

em.emit('firstEvent',"hello")
setTimeout(() => em.emit('secondEvent',10), 10)
```

6.2. Exercice

Dans cet exercices vous allez devoir importer et exporter au format csv des données similaire à celles de l'API de la Disponibilité en temps réel des places et des vélos dans les stations vélos en libre-service Naolib de Nantes Métropole.

6.2.1.

6.2.2. Mettre en place les jeux de tests

Créer un package.json et ajouter

[3]source : https://www.fil.univ-lille.fr/~routier/enseignement/licence/jsfs/html/node-asynchrone.html

```
"type": "module"
```

, les .js seront considérés comme des modules, ajouter également dans scripts

```
"test": "node --test --experimental-test-coverage test/*"
```

les fichiers du répertoire test se terminant par `.test.js` seront exécutés.

6.2.3. Compléter la classe stations

La méthode add et définir des test pour remove.

6.2.4. Utiliser une bibliothèques csv en événementielle

Utiliser la bibliothèque csv de votre choix pour importer et exporter des stations, les données se trouvent dans le répertoire data.

Chapitre 3. Un service proxy

1. Cours

1.1. HTTP

Vous avez déjà vu le protocole HTTP en cours mais voici quelques rappels, le protocole HTTP est un protocole applicatif (niveau 7), client/serveur, sur TCP.

La requête est la réponse sont constitués d'une entête et qu'éventuellement d'une réponse. Voici, un exemple d'échange applicatif avec une API de l'opendata de Nantes. Les échanges étant applicatif, la connexion ne fait pas partie de la trace.

```
#requette sans body
GET https://data.nantesmetropole.fr/api/explore/v2.1/catalog/datasets/244400404_disponibilite-temps-reel-

#reponse
HTTP/2 200 OK
server: openresty
date: Sun, 01 Feb 2026 16:56:23 GMT
content-type: application/json; charset=utf-8
x-ratelimit-remaining: 9980
x-ratelimit-limit: 10000
x-ratelimit-reset: 2026-02-02 00:00:00+00:00
cache-control: no-cache, no-store, max-age=0, must-revalidate
content-security-policy: upgrade-insecure-requests; default-src 'self' https://*.screeb.app; script-src '
x-frame-options: DENY
vary: Accept-Language, Cookie, Host
content-language: fr-fr
access-control-allow-origin: *
access-control-allow-methods: POST, GET, OPTIONS
access-control-max-age: 1000
access-control-allow-headers: Authorization, X-Requested-With, Origin, ODS-API-Analytics-App, ODS-API-Ana
access-control-expose-headers: ODS-Explore-API-Deprecation, Link, X-RateLimit-Remaining, X-RateLimit-Limi
x-datadog-trace-id: 16988571825955757117
strict-transport-security: max-age=31536000;includeSubdomains
x-xss-protection: 1; mode=block
x-content-type-options: nosniff
referrer-policy: strict-origin-when-cross-origin
x-ua-compatible: IE=edge
permissions-policy: midi=(),microphone=(),camera=(),magnetometer=(),gyroscope=(),fullscreen=(self),paymen
content-encoding: gzip
x-http2-stream-id: 3
transfer-encoding: chunked

{
  "total_count": 126,
  "results": [
    {
      "last_update": "2026-02-01T16:56:00+00:00",
      "available_bike_stands": "0",
      "number": "24",
      "available_bikes": "15",
      "name": "VERSAILLES",
      "address": "25, quai de Versailles - Ile de Versailles",
      "rental_methods": null,
      "bike_stands": 15,
      "position": {
        "lon": -1.553712,
        "lat": 47.224515
      }
    },
```

Le protocole dispose de verbes ou de méthodes, nous utiliserons les méthodes suivantes :

GET (READ)

permet de demander une ressource.

POST (CREATE)

permet d'envoyer des informations vers une ressource pour un traitement.

PUT (UPDATE)

permet de remplacer ou d'ajouter une ressource sur le serveur. L'URI fourni est celui de la ressource en question.

DELETE

permet de supprimer une ressource

La convention pour les ressources :

Tableau 3.1. Convention pour les ressources

Chemin	Ressource	Méthode	Body	Code HTTP2	Sens
/station	Les stations (une collection)	GET		200 (OK)	Donne la liste des stations
/station/{id}	Une station	GET		200 (OK) ou 204 (No Content) ou ..	Donne une station si elle existe
/parking	Les stations (une collection)	POST	Le JSON décrivant un parking	201 (CREATED) ou ..	Permet de créer une station
/parking/{id}	Une station	PUT	Le JSON décrivant la modification	200 (OK)	Sens modifier une station en connaissant son id.

Vous trouverez ici les codes HTTP : https://developer.mozilla.org/en-US/docs/Web/HTTP/Status.

1.2. MongoDB

MongoDB est un SGBD orienté document et ne nécessitant pas de schéma prédéfini des données. Vous trouverez ici la documentation : `https://www.mongodb.com/docs/drivers/node/current/`.

Dans un soucis de simplification nous utiliserons une version mémoire de mongo : mongodb-memory-server.

Un exemple de CRUD :

```
'use strict'
import { MongoClient } from 'mongodb'
import { MongoMemoryServer } from 'mongodb-memory-server';

//connexion

const mongod = await MongoMemoryServer.create();
const uri = mongod.getUri();
const client = new MongoClient(uri)

try {
    //acces à la BD maBD
    const maBD = client.db("maBD")
    //acces à la collection
    const maCollection= maBD.collection("maCollection")
    //Dans mongodb les schémas ne sont pas obligatoires, cet exemple
    //ne les utilsent pas
```

```
    //un document
    const monDocument = {
            title: "Record of a Shriveled Datum",
            content: "No bytes, no problem. Just insert a document, in MongoDB",
        }
    //insertion /!\ le parametre est modifie
    let result = await maCollection.insertOne(monDocument)
    console.log(`I-Un document inséré avec l' _id: ${result.insertedId}`)

    //lecture
    const requete = {title: "Record of a Shriveled Datum"};
    const options = {
         projection: {_id:0,  title: 1, content: 1 },
    };
    const data = await maCollection.findOne(requete, options);
    console.log('findOne : ',data);

    //update
    const updateDoc= {$set: {title: "Enregistrement d'une donnée ratatinée"}}
    result = await maCollection.updateOne(requete, updateDoc, options);
    console.log('update count:',result.modifiedCount)
    //findAll
    const curseur = maCollection.find({})

    // parcours
    for await (const undoc of curseur) {
        console.log(undoc);
    }
    //ou await curseur.toArray()

    //Suppression d'un document
    await maCollection.deleteOne({title: "Enregistrement d'une donnée ratatinée"});
    //suppression de tous les documents
    await maCollection.deleteMany({});
} finally {
        await client.close();
        await mongod.stop();

}
```

MongoDB permet également de créer des schémas `https://www.mongodb.com/docs/atlas/app-services/schemas`, les schémas sont écris en BSON une extension typée de JSON. Voici une collection avec validation :

```
db.createCollection("students", {
   validator: {
      $jsonSchema: {
         bsonType: "object",
         title: "Student Object Validation",
         required: [ "address", "major", "name", "year" ],
         properties: {
            name: {
               bsonType: "string",
               description: "'name' must be a string and is required"
            },
            year: {
               bsonType: "int",
               minimum: 2017,
               maximum: 3017,
               description: "'year' must be an integer in [ 2017, 3017 ] and is required"
            },
            gpa: {
               bsonType: [ "double" ],
               description: "'gpa' must be a double if the field exists"
            }
         }
      }
   }
```

```
} )
```

Mongoose est une sur couche de mongoDB qui est un Object Document Mapping qui simplifie l'utilisation du SGBD, voici le même exemple :

```
'use strict'
import mongoose from 'mongoose';
import { MongoMemoryServer } from 'mongodb-memory-server';

//connexion

const mongod = await MongoMemoryServer.create();
const uri = mongod.getUri();
//connexion sur mabd
await mongoose.connect(mongod.getUri(),{dbName: "maBD"})
//Dans mmongoose les schema sont obligatoires

try {
    const monSchema = new mongoose.Schema({
        title: String,
        content: String
    });

    const monModel = mongoose.model('document', monSchema);
    const document = new monModel(
        {title: "Record of a Shriveled Datum",
        content: "No bytes, no problem. Just insert a document, in MongoDB"});
    //insertion /!\ le document est modifié
    await document.save()
    console.log("insertion",document) //l'objet a été modifié
    const requete = {title: "Record of a Shriveled Datum"};
    //lecture
    const projection = { _id: 0, __v:0 }
    console.log("findOne",await monModel.findOne(requete,projection))

    //update
    //save repose sur les midleware et la validation
    //updateMany Non
    document.title="Enregistrement d'une donnée ratatinée"
    await document.save();

    //findAll
    console.log(await monModel.find({},projection))

    //Suppression d'un document
    await monModel.deleteOne({title: "Enregistrement d'une donnée ratatinée"});
    //suppression de tous les documents
    await monModel.deleteMany({});
    console.log(await monModel.find({},projection))
}finally {

   await mongoose.disconnect();
   await mongod.stop();
}
```

1.3. Express

Express est un framework web minimaliste, vous en trouverez la documentation ici : `https://expressjs.com`.

Pour installer express : `npm install express`.

Express repose sur l'utilisation de midleware. Les middlewares sont des fonctions qui s'exécutent lors de la requête au serveur. Ces fonctions ont accès aux paramètres de la requête et de la réponse.

Voici une exemple de d'utilisation d'une route en get sans midleware :

```
import express from 'express';

const app = express()
```

```
app.listen(3000, () =>
    console.log('Example app listening on port 3000!'),
)

app.get('/about', (req, res) => {
    console.log('url',req.url)
    console.log('params',req.params)
    console.log('query',req.query)
    return res.send(`/about Page : url:
        ${req.url} params ${JSON.stringify(req.params)} query ${JSON.stringify(req.query)}`)
})

app.get('/about/:user', (req, res) => {
    console.log('url',req.url)
    console.log('params',req.params)
    console.log('query',req.query)
    return res.send(`/about/:user Page : url:
        ${req.url} params ${JSON.stringify(req.params)} query ${JSON.stringify(req.query)}`)
})
```

Vous pouvez tester cet exemple avec différents paramètres.

Une version avec un midleware :

```
const app = express();

app.listen(3000, () =>
    console.log('Example app listening on port 3000!'),
);

const logger  = (req, res, next) => {
    console.log(req.url)
    console.log(req.params)
    console.log(req.query)
    next()
}
app.use(logger)

app.get('/about', (req, res) => {
    return res.send('About Page')
})
```

Lorsque l'utilisateur visite "/about" Express exécute le `app.get()`. Mais juste avant, il exécutera le middleware spécifié dans `app.use()`. Une fois le middleware exécuté, `next()` continuera l'exécution de `app.get()`.

L'exemple suivant est un crud sur une collection de user :

```
'use strict';

import express from 'express';

//notre modèle un Utiilisateur et l'ensemble des utilisateurs
//les utilisateurs sont identifiés par leur login
class User {
    login
    password
    constructor(obj) {
        Object.assign(this,obj)
    }
}
class Users {
    users = []
}

//pas de bd uniquement en mémoire
const model = new Users()

//notre DAO
const userDao = {
```

```
    //tous les utilisteurs
    findAll : () => model.users,

    //ajout un utilisateur
    //renvoie l'utilisateur ajouté ou null sinon
    findByLogin : (login) => {
        const users = model.users.filter((user)=>user.login==login)
        return users.length!=0 ? users[0]:null
    },
    //supprime un utilisateur
    //renvoie l'utilisateur supprimé ou null sinon
    deleteByLogin: (login) => {
        const user = userDao.findByLogin(login)
        if (user==null)
            return null
        model.users = model.users.filter((user)=>user.login != user.login)
        return user
    },
    //ajout un utilisateur
    //renvoie l'utilisateur ajouté ou null si il était déjà présent
    add: (user) => {
        const userByLogin = userDao.findByLogin(user.login)
        if (userByLogin != null)
            return null
        model.users.push(user)
        return user
    },

    //Modifie un utilisateur
    //premd en paramètre le login du user à modifier et la modification
    //renvoie le user modifier ou null
    update: (login, user) => {
        const userByLogin = userDao.findByLogin(login)
        if (user==null)
            return null
        userDao.deleteByLogin(login)
        userDao.deleteByLogin(user.login)
        userDao.add(user)
        return user
    }

}
//la methode qui permettra de lancer le serveur
const init = async () => {
    //Un serveur qui écoute sur le port 3000
    const app = express();
    app.listen(3000, () =>
        console.log('Example app listening on port 3000!'),
    );
    app.use(express.json()) // parse json body content

    const router = express.Router()

    //une route est définie par
    //son verbre
    //son path

    app.get('/user',(req, res)=>res.status(200).send(userDao.findAll()))
    app.post('/user',(req,res) => {
            console.log(req.body)
            const userToAdd = req.body
            const user = userDao.add(userToAdd)
            if (user != null)
                res.status(201).send(user)
            else
                res.status(400).send({message: 'already exist'})
        })

    app.get('/user/:login', (req,res)=>{
        console.log(req.params)
```

```
        const user = userDao.findByLogin(req.params.login)
        if (user!=null)
            return res.status(200).send(user)
        else
            return res.status(404).send({message: 'not found'})
    })
    app.delete('/user/:login',(req,res)=>{
        const user = userDao.deleteByLogin(req.params.login)
        if (user!=null)
            res.status(200).send(user)
        else
            res.status(404).send({message: 'not found'})
    })
    app.put('/user/:login',(req,res)=>{
        const user = userDao.update(req.params.login,req.body)
        if (user!=null)
            res.status(200).send(user)
        else
            res.status(404).send({message: 'not found'})
    })

};

//evenement capturé en cas d'erreur
process.on('unhandledRejection', (err) => {

    console.log('err',err);
    process.exit(1);
});
init();
```

2. Exercices

2.1. Exercice 01

Vous disposez d'un serveur REST réalisé en GO, lancez le avec **go run main.go**. En utilisant obtenez la liste des livres, le livre d'Id 1. Quel sont les codes de retour, si le livre n'existe pas ou si l'Id n'est pas un entier.

Ajouter un livre, modifier et supprimer un livre.

Que pensez-vous de la robustesse de cette API ?

2.2. Exercice 02

En partant des exemples précédents coder et tester un DAO simple pour les stations :

```
const dao(collection) = {
return {
    findAll: async ()=> {
        throw new Error("Must be implemented")
    },
    findOne: async(id)=> {
        throw new Error("Must be implemented")
    },
    delete: async(id) => {
        throw new Error("Must be implemented")
    },
    deleteAll: async() => {
        throw new Error("Must be implemented")
    },
    add: async(obj) => {
        throw new Error("Must be implemented")
    },
    update: async(id,obj) => {
        throw new Error("Must be implemented")
    }
```

```
}}
```

2.3. Exercice 03

Reprendre l'exemple du cours (`01_serveur_en_memoire.mjs`) et le tester avec curl ou un HTTP Request, si vous utilisez IntelliJ ou https://hoppscotch.io/ ou tout autre client de votre choix. Essayer d'obtenir un comportement inattendu quelles sont les faiblesses de notre code ?

2.4. Exercice 04

En partant du squelette fourni, vous allez construire une API Rest sur des utilisateur (User) définis par leur login et leur mot de passe (password). Des jeux de tests dans test_todo, vous sont fournis, ils devront être déplacés dans test pour chacune des étapes. L'architectures des fichiers est :

```
.env //fichier contenant les constantes de l'application

.package.json
// qui permet de lancer les tests avec npm test
// de lancer le serveur avec npm start
// de lancer la génération de la documentation avec npm run start-endoc

serveur.mjs //le serveur qui lance l'application

app.mjs //l'application qui enregistre les routes

api //les fichiers de l'application
  controlleur //les controleurs
  dao //les dao
  model //les modèles
  route //les routes
test // les test à lancer
test_todo //les fichiers à déplacer dans test
```

Les dépendances suivantes sont utilisées :

dotenv

: `https://www.npmjs.com/package/dotenv` pour charger des variables d'environnement depuis un fichier.env, ne pas oublier de le créer si il n'est pas présent

express

: `https://www.npmjs.com/package/express` un framework web léger

mongodb

: `https://www.npmjs.com/package/mongodb` des drivers pour mongodb

mongodb-memory-server

: `https://www.npmjs.com/package/mongodb-memory-server` un serveur mongodb travaillant en mémoire, utilisé pour les tests

mongoose

: `https://www.npmjs.com/package/mongoose` un Object Data Modeling

nodemon

: `https://www.npmjs.com/package/nodemon` qui permet de relancer le serveur si les fichiers sont modifiés

supertest

: `https://www.npmjs.com/package/supertest` pour les tests HTTP

swagger-autogen

: `https://www.npmjs.com/package/swagger-autogen` pour la génération automatique de documentation à partir des routes et d'annotations

swagger-ui-express

: `https://www.npmjs.com/package/swagger-ui-express` utilisé par swagger-autogen

2.4.1. Le modèle

Le fichier de test à utiliser est model.test.mjs, déplacer le dans test est compléter la classe User dans userModel.mjs.

2.4.2. Le DAO

Le fichier de test à utiliser est dao.test.mjs, déplacer le dans test est compléter l'objet userDAO dans userDAO.mjs.*Si vous rencontrez un problème de version sur votre VM de version pour mogo-memory_server définissez sur votre VM la variable MONGOMS_VERSION* `export MONGOMS_VERSION=7.0.3`.

2.4.3. Le contrôleur

Cette fois le code du contrôleur est fourni userConrolleur.mjs à vous de fournir les jeux de test.

2.4.4. L'application

Le fichier de test à utiliser est app.test.mjs, déplacer le dans test est compléter route.mjs.

2.4.5. La documentation

Générer et tester la documentation, swagger-autogen.mjs en permet la configuration mais n'est pas suffisant, il vous faudra égalment ajouter des annotations. La documentation sera accessible via /doc (`app.use('/doc', swaggerUi.serve, swaggerUi.setup(swaggerJson))`).

2.4.6. Pour aller plus loin, la validation des entrées

Notre routeur, ne vérifie pas les paramètres en entrée, vous pouvez améliorer cela avec express-validator (`https://www.npmjs.com/package/express-validator`).

2.5. Exercice 05

En partant du code fourni qui réalise la synthèse des exercices précédents implémenter les routes suivantes :

/station

: La liste des stations

/station/near/:lat/:lon

: Les stations triées par distance par rapport à la latitude et la longitude

/station/available-bikes

: Les stations avec des vélos disponibles

/station/available-bikes/near/:lat/:lon

: Les stations avec des vélos disponibles triées par distance par rapport à la latitude et la longitude

Pour optimiser le temps de réponse, vous vérifierez pour chaque requête si la dernière mise à jour est de moins de 10 minutes, si c'est le cas vous répondrez avec ce qui est en BD sinon vous mettrez à jour votre BD avec les données de l'opendata de Nantes.

Le DAO stationFetchDAO permet de consulter l'opendata. Le DAO de station contient la méthode addMany qui permet d'insérer une collection de stations.

Le travail de proxy est à réaliser dans le contrôleur station en utilisant la variable lastUpdate

```
const doUpdate =
            lastUpdate == null ||
            new Date().getTime() - lastUpdate.getTime() > 1000 *60*10
```

Les tests modèles, DAO, contrôleurs, applications ne sont pas suffisants, vous pouvez les augmenter.

Chapitre 4. Un deuxième service avec authentification

Dans cette partie, vous pouvez partir de l'exercice sur les utilisateurs (04-mvc) pour permettre à un utilisateur de gérer ses stations favorites, dans un premier temps sans authentification, pour avec une authentification JWT (JSON Web Token).

Vous devrez pouvoir lister les stations favorites, ajouter et supprimer une station favorite, c'est opération n'étant possible que si l'authentification a réussie.

Pour JWT, de nombreuse bibliothèque ou midleware existent : `https://www.npmjs.com/search?q=jwt`.

Chapitre 5. Le point d'entrée de notre application

Masquer vos deux premiers services derrière un point d'entrée unique sur le port 80 pour répondre au scénario de votre choix. N'oubliez pas de fournir une documentation.